```
    context.dispatch({
      type: "setTicketsToBuy",
      amount: parseInt(target.value, 10),
    })
  }

  return (
    <div>
      <Header>How many tickets would you like?</Header>
      <span className="select">
        <select onChange={setTicketsOnChange}>
          {options(context.state.seatsPerRow)}
        </select>
      </span>
    </div>
  )
}

export default VenueHeader
```

This component has changed some. It takes no props, for one. The first thing we do in the component function itself is call a new hook called `useContext`: `const context = React.useContext<IsVenueContext>(VenueContext)`.

The argument to `useContext` is the name of a context whose provider has this component in scope. The return value is the context itself. By using this hook we guarantee that the component will be refreshed if the context changes—just as if the data in the context had been passed as props, but without the complexity of actually passing them.

The rest of the component has changed slightly. When we pass the number of seats in a row to generate the list of options, we now take that number from `context.state.seatsInRow` rather than from the props. The `setTicketsOnChange` handler triggers a call to the reducer using `context.dispatch`. The dispatch call changes the state, which means the context has changed and which triggers a redrawing of all the components that use the context.

VenueBody

The `VenueBody` component still largely creates a bunch of rows but now also uses